

一个合理的工具链配置流程

(本文仅针对重庆大学CRTC2026校内赛撰写)

一些问题

当你准备接触嵌入式，参加校内比赛或是自己做一些东西时候，可能会选择向有经验的高年级同学选择求助，除了部分愿意帮助你进行线下配置的大好人，大部分高年级同学还是倾向于选择甩一个b站链接或者直接让你搜索相关视频，例如：

- Keil v5+MDK-ARM注册机破解教程
- Visual Studio Code与Keil工具链配置教程
- STMCubeIDE配置教程

然后就让大家自己去尝试学习配置相关软件时，更多时候作为一个新入学的嵌入式新人，在你过去十几年求学生涯中，90%的人可能并没有被引导学习过相关的知识，于是必然需要头铁去啃那些的视频，更不必提用百度网盘，QQ群文件等下载一堆来路不明的软件。就我个人看来，这种方式是极其不合理的，虽然在一定程度上筛选了一批对于PC操作不敏感的学生，但是也断绝了一部分对于理论设计有意识的人的热情。

因此为了抹平配置问题这一壁垒，对于重庆大学千里战队的梯队成员，抑或是参与校内相关比赛前来学习的非队员同学甚至是校外人士，我们立志提供一份有别于网络上其他同质化教程的快捷配置方案。

一套办法

作为一名合格的大学生，通过正规手段获取生产工具是一项必要的技能。

也许大家看过诸如“付费解锁”“限时秒杀”WinRAR，VScode等软件的梗图，但是在这里我们一定要达成的共识是，本科阶段学习过程中你使用到的90%的工科软件都有免费获取的途径，因此在下文我们会引导大家怎么去下载这一内容。



注册重庆大学学生邮箱

作为重庆大学的学生，带有“学号+stu.cqu.edu.cn”格式的官方学生邮箱是能够辅助证明学生资质的优质邮箱，普遍用于专业软件领取，教育优惠申请，学生资质证明。那么我们去注册这一邮箱呢？

首先，我们需要访问重庆大学网上服务大厅：<https://i.cqu.edu.cn/index.html#/>，并且使用我们的统一身份认证进入该网站。需要注意的是，如果你从该文档的链接直接选择跳转，由于没有经过统一身份认证，你会遇到406的错误，这表示后端返回的结果前端无法解析，只需要将 `index.html#/` 这部分内容手动删去再次访问即可。



成功进入后，点击上图中箭头所指示的电子邮箱申请，跳转到Widedu代理的邮箱申请网站并且选择新增邮箱申请。

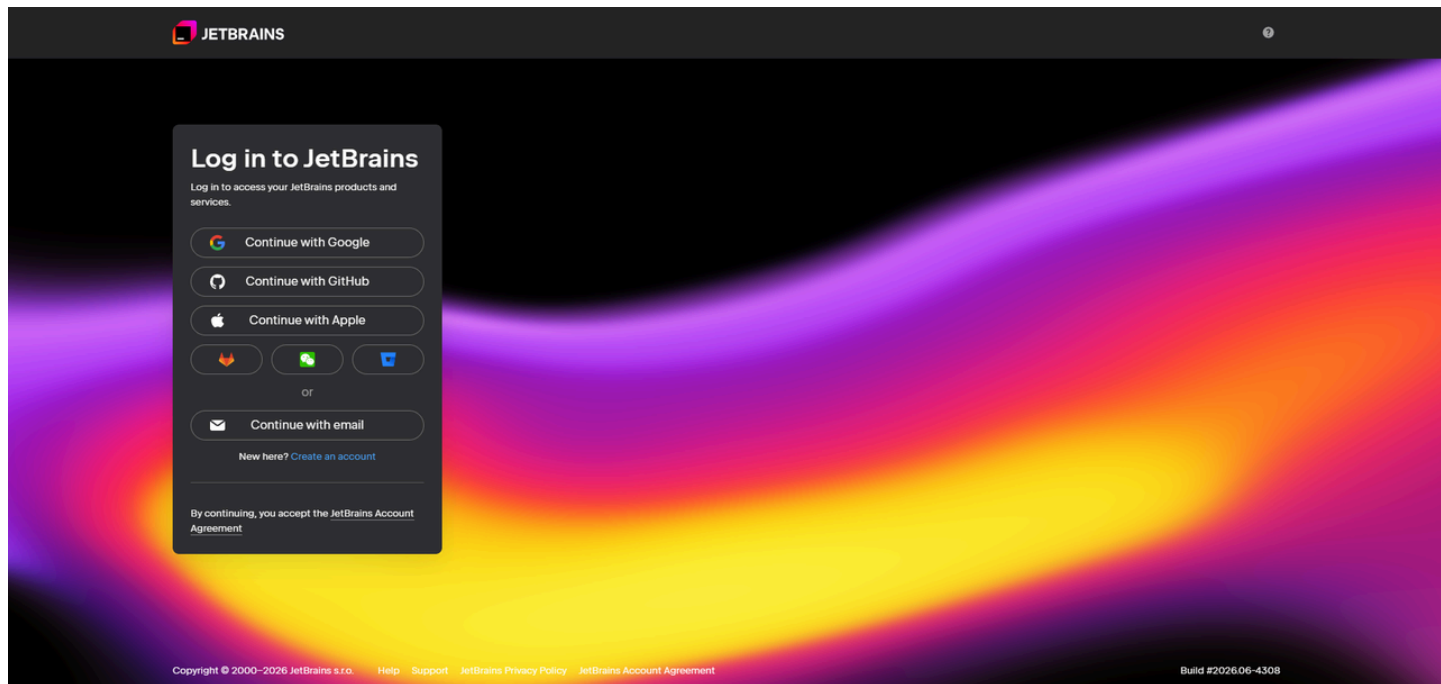


按照要求填写相关信息后等待邮箱下发，这一过程大概会等待1~2天的时间。

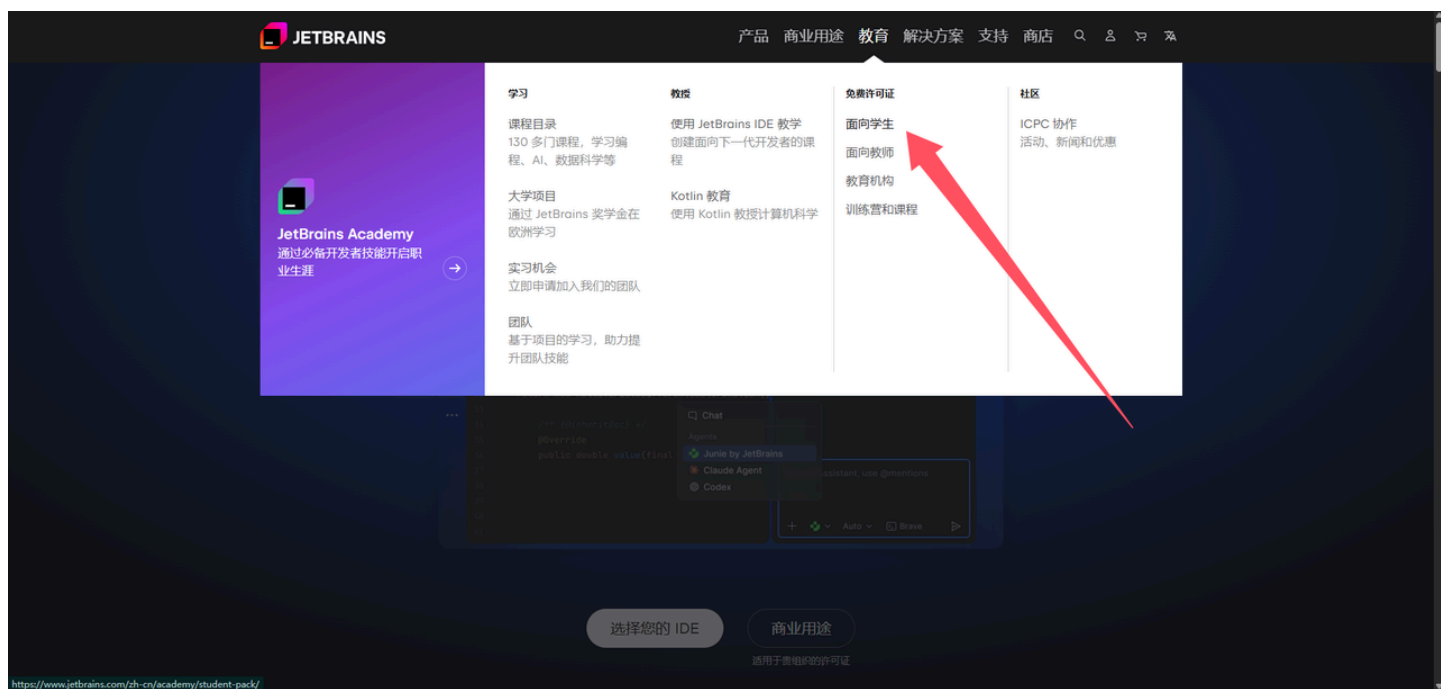
而对于非重庆大学学生，可以考虑自己本校的邮箱申请服务；对于非在校人士，可以考虑使用非商业用途的免费个人版或者破解版。

登录JetBrains国区网站并且申请教育许可证

搜索JetBrains，或者直接访问<https://www.jetbrains.com/zh-cn/>，在右上角选择你最常用的登录方式，在这里个人推荐email≈Google>GitHub。



接下来，我们在上方找到“教育”→“面向学生的免费许可证”



点击下图的“此处”进入引导流程：



JetBrains Student Pack

用于学习和工作准备的一体化开发工具包

学生可以免费使用所有专业开发者工具，包括 IntelliJ IDEA、PyCharm 等。

申请前，请先在此处确认您的申请资格和申请方式，然后按照指导提交申请。



最后点击“教育授权申请”跳转到申请界面按照要求填写相关信息即可。

通过有效的大学电子邮件地址申请 JetBrains 教育授权

操作步骤：

- 1 前往 **JetBrains** 官网的[教育授权申请入口](#)。
- 2 以您本人的真实学籍状况和姓名填写申请表格，申请方式一行选择“大学电子邮件地址”。

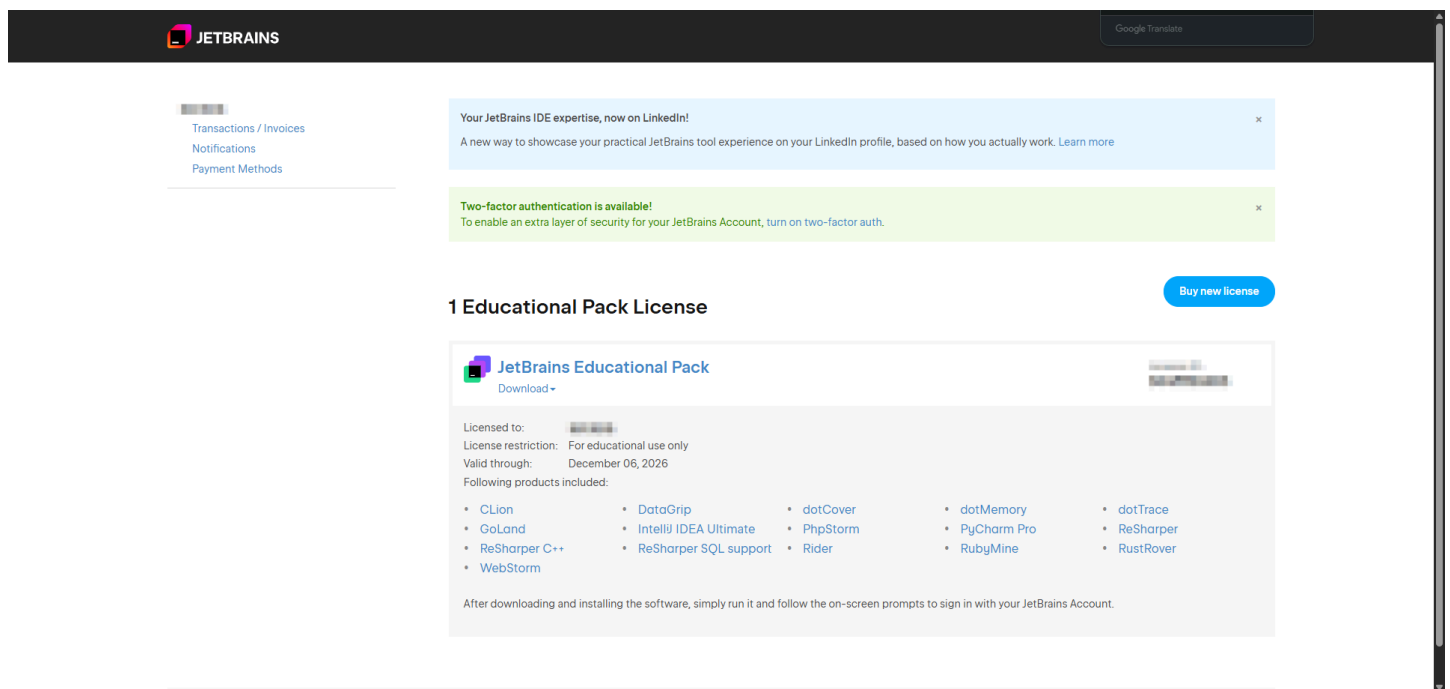


如申请没有成功提交，页面会停留在表单并提示问题原因，此时请依照表单上的提示信息对应补充后重新提交。

如遇到提示电子邮件不接受的情况，请在申请方式一行改选择通过“官方文件”进行申请。

- 3 提交申请成功，同时您会收到一个系统自动推送的邮件，协助您完成许可证的开通。

通过这一步骤成功申请到教育许可证后，我们可以在个人界面看到如下信息：



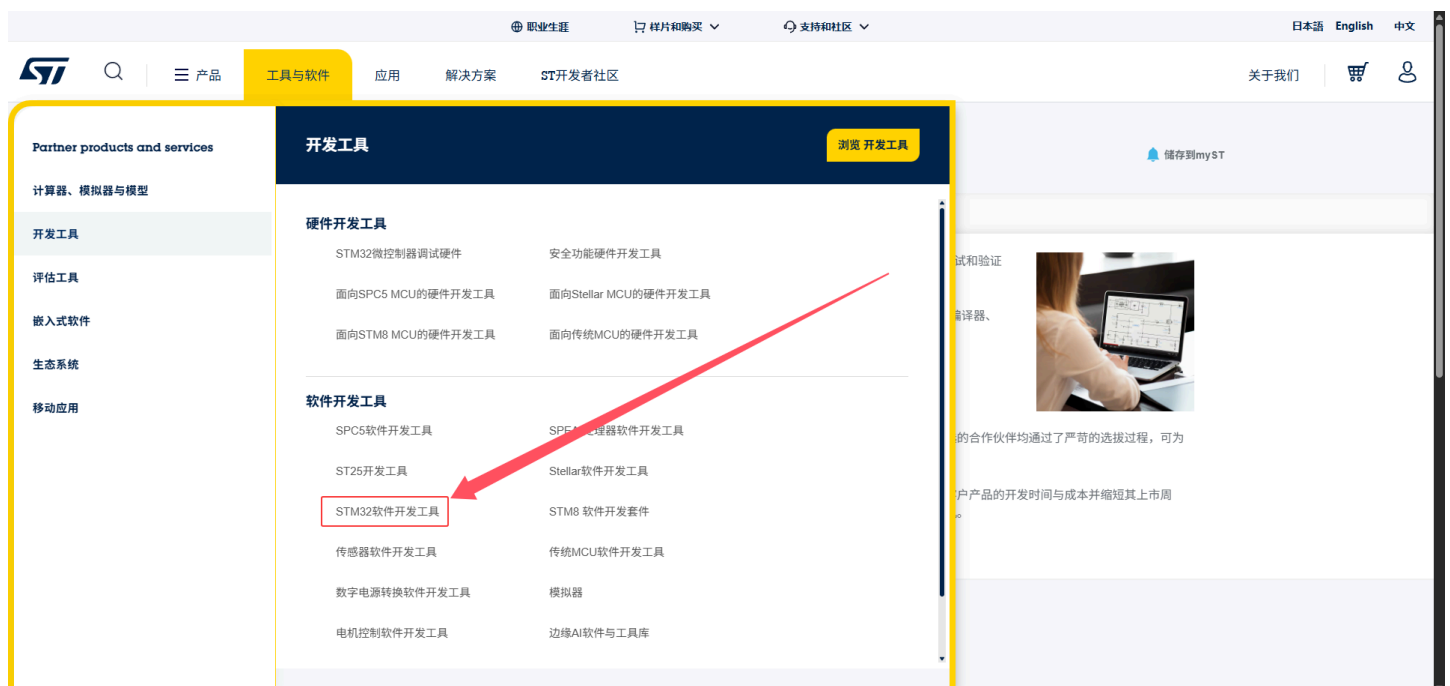
与此同时我们也可以并行下载CLion，一般来说我们使用的都是面向个人的免费版本，这一版本在实际功能上存在一定的阉割，因此直接点击右下角便可导入我们刚才申请的教育许可证，享受完整的IDE内容。

下载STM32系列工具链

这一部分所有的内容均来自于ST意法半导体公司的中文官网

https://www.st.com.cn/content/st_com/zh.html

现在上方“工具与软件”栏中找到“开发工具”-->“软件开发工具”-->“STM32软件开发工具”，



然后在这里“按名称或描述检索工具与软件”进行我们的下一步资源收集工作。

职业生涯

样品和购买

支持和社区

日本語English中文

ST

产品

工具与软件

应用

解决方案

ST开发者社区

关于我们

购物车

用户

开发工具硬件

STM32软件开发工具

保存到myST

概述

产品选择器

文件

工具与软件

All tools & software types

Minify

按名称或描述检索工具与软件

按名称或描述检索工具与软件

只显示由ST供应的产品

重置

87 total entries

产品编号	Description	供应商	Licence type	支持的设备
B-STLINK-VOLT	Voltage adaptation board for STLINK-V3SET	ST	Free	STM32
B-U585I-IOT02A	Discovery kit for IoT node with STM32U5 series	ST	Free	STM32L4
FP-AI-FACEREC	Artificial Intelligence (AI) face recognition function pack for STM32Cube	ST	Free	
FP-AI-MONITOR1	Artificial Intelligence (AI) face recognition function pack for STM32Cube	ST	Free	

STM32CubeMX

首先是我们进行芯片配置的最直接软件

职业生涯

样品和购买

支持和社区

日本語English中文

ST

产品

工具与软件

应用

解决方案

ST开发者社区

关于我们

购物车

用户

F2 F4 N5 F7 N7 N7U5 N8

Midstream MCUs

S0 C0 F0 F1 F3 G4

Ultra-low-power MCUs

L0 L0 L1 L4 L4+ L5 U3 U5

Wireless MCUs

W80 WL WL3 WB WBA MPU MP2

CS

获取软件

软件名称	支持的硬件	ECCN	操作系统	版本	操作
STM32CubeMX	All MCU/MPU except STM32C5 Series	NEC (EU) 5D992.c (US)	Windows	6.18.0 最新版本	下载
STM32CubeMX2	STM32C5 Series	NEC (EU) 5D992.c (US)	Windows	1.0.1 最新版本	下载

STMicroelectronics recommends always keeping your software up to date

Featured Products

New STM32C5 Cortex-M33 MCUs starting at \$0.64

STM32WBA6系列

STM32H7S3

这里我们需要选择下载的版本为最新版。

STM32CubeMonitor

最新即可，不多做赘述。

该文件主要用于实时监测中的可视化曲线生成。

STM32CubeProgrammer

最新即可，不多做赘述。

该软件主要用于ST-Link的更新，高级设置以及烧录。

STM32CubeCLT

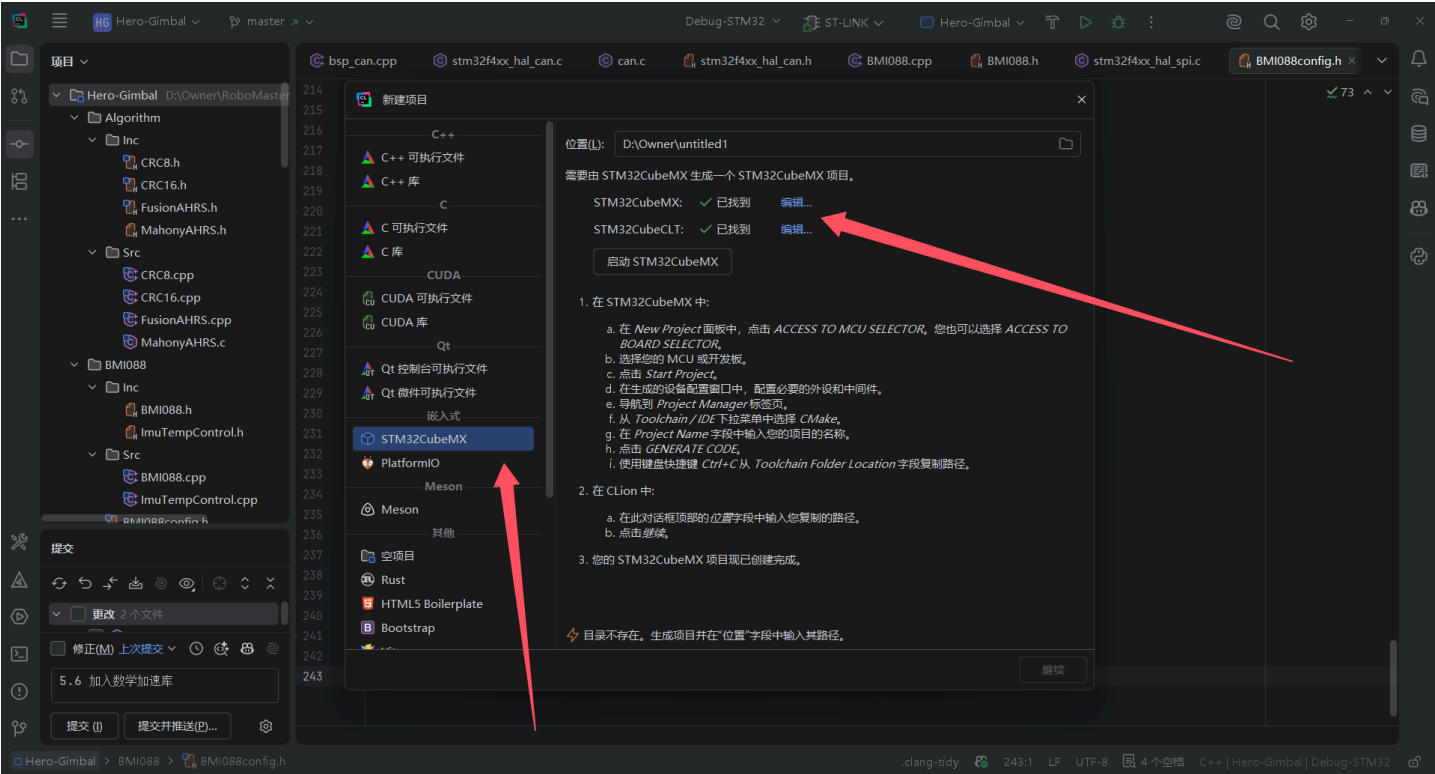
这是STM32系列工具包，主要用于环境配置，它的位置与上述三者稍有不同：



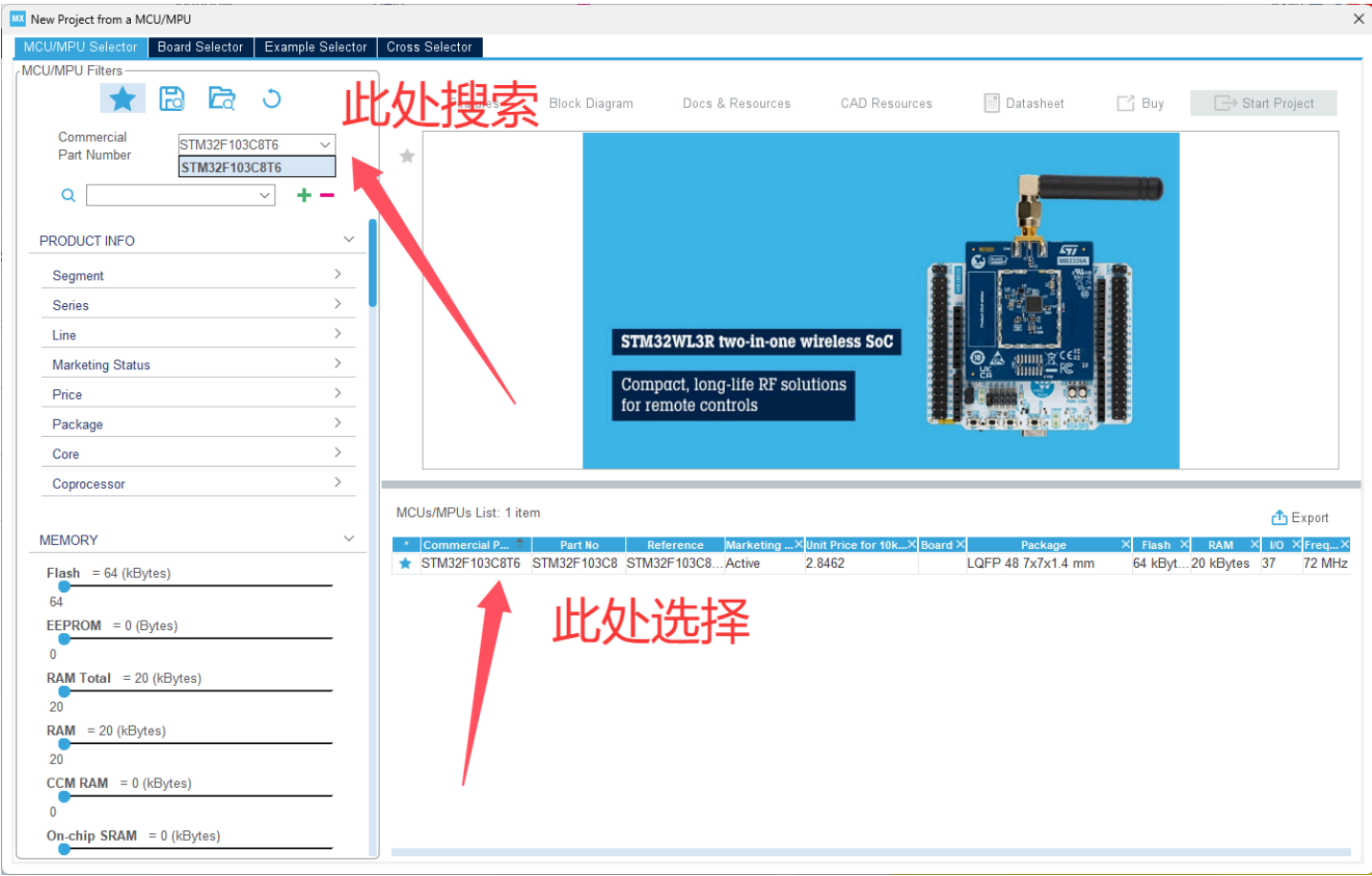
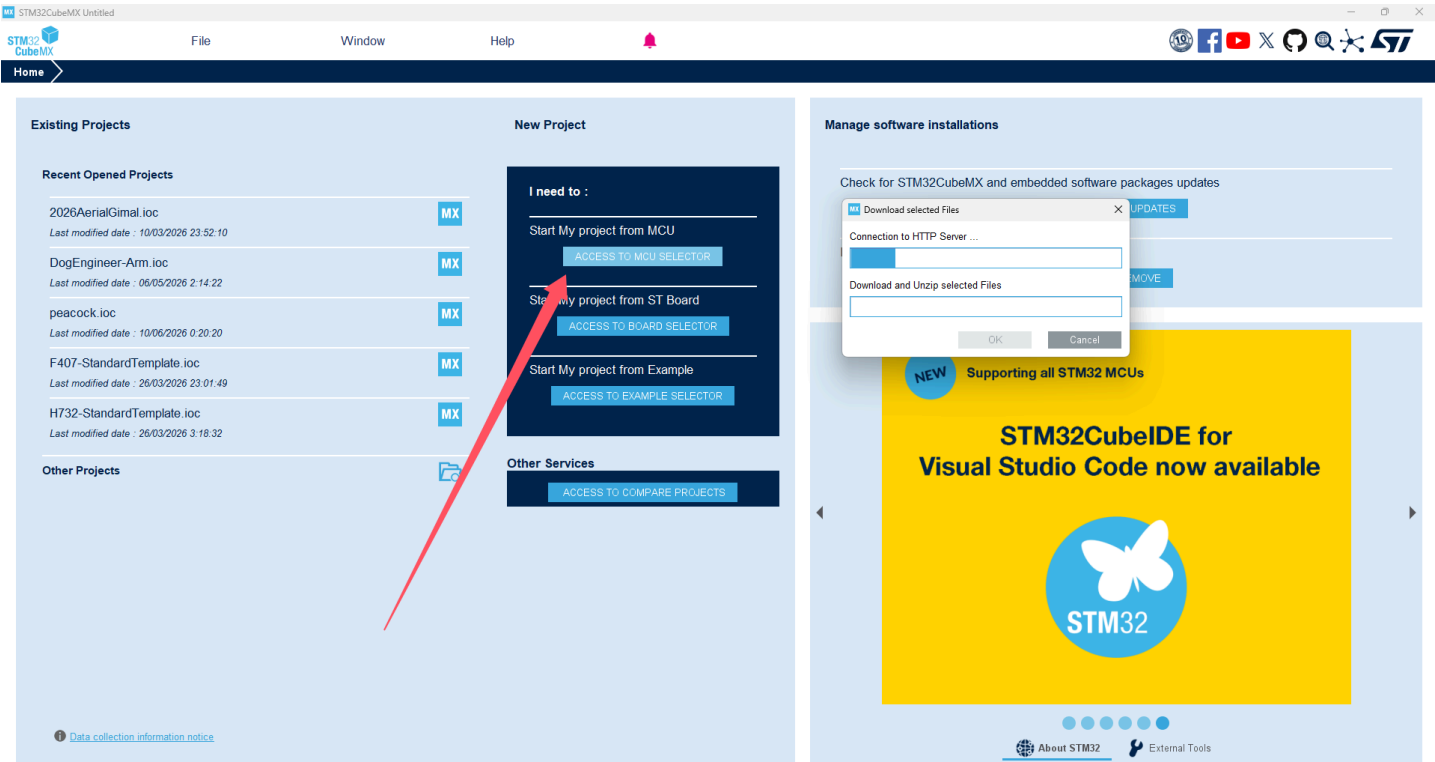
同样下载最新版本即可。

配置CLion+STM32CubeMX工具链

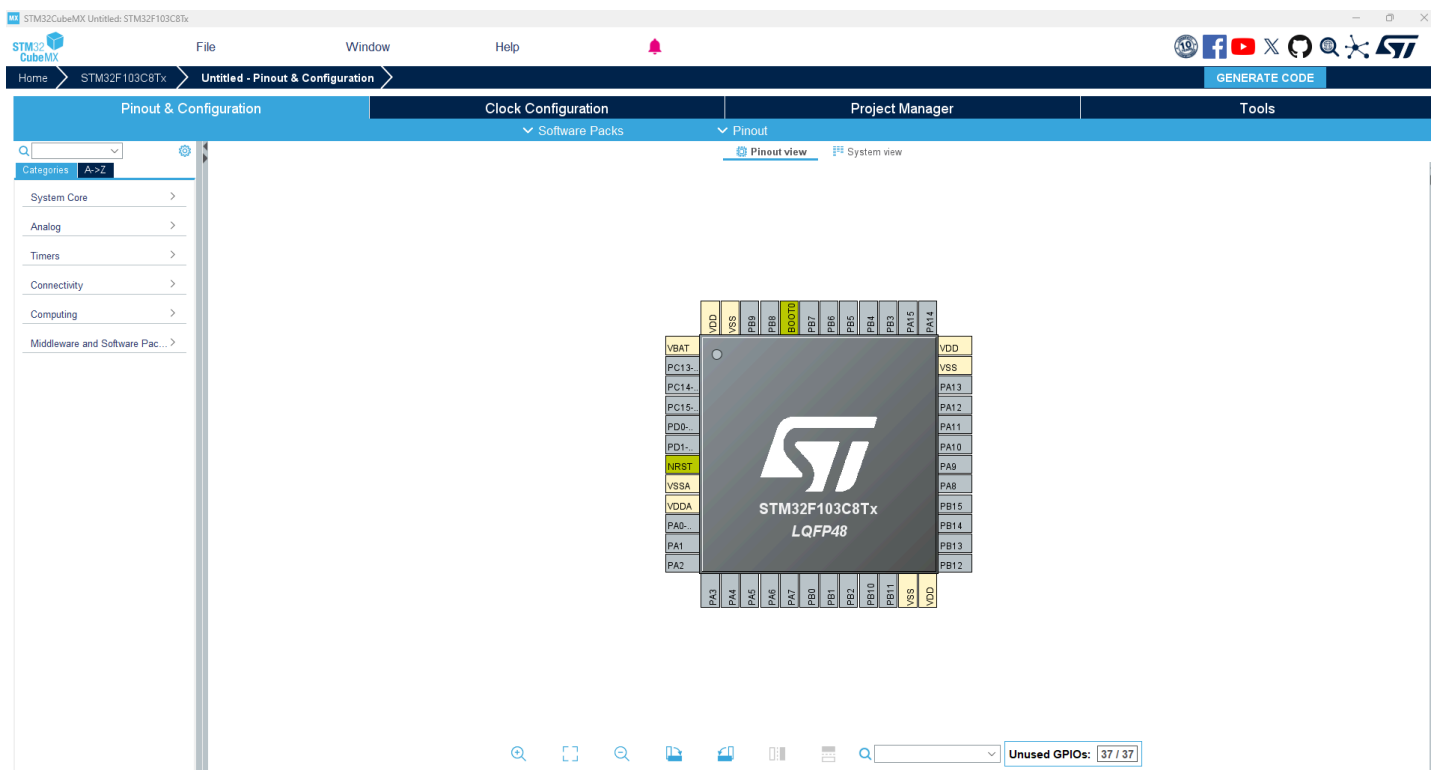
打开CLion，创建新项目，并且选择嵌入式中的STM32CubeMX，确保IDE能够找到STM32CubeMX与STM32CubeCLT的位置。



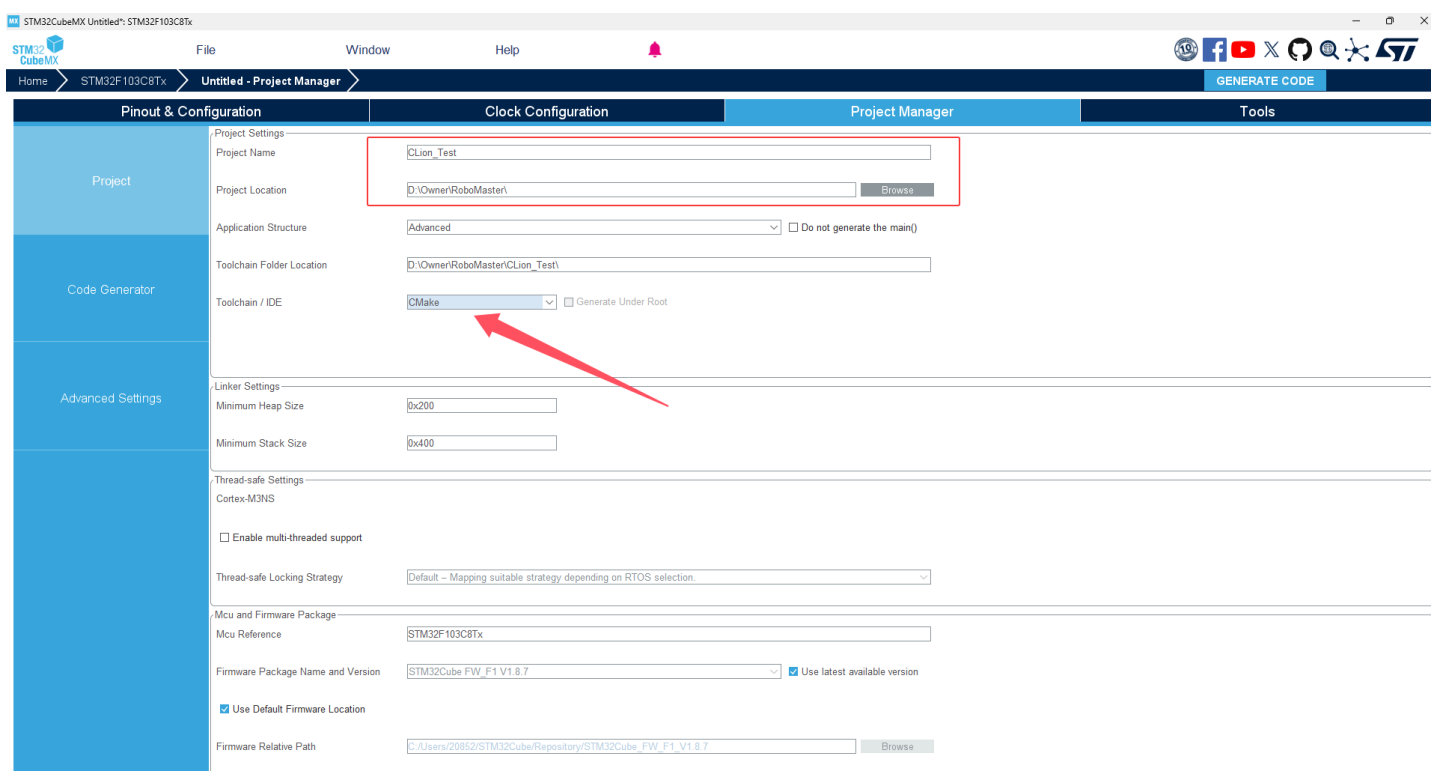
此处我们以STM32CubeMX中的STM32F103C8T6为例，在“ACCESS TO MCU SELECTOR”



稍后进入如下界面后，选择上方栏目中的“Project Manager”。

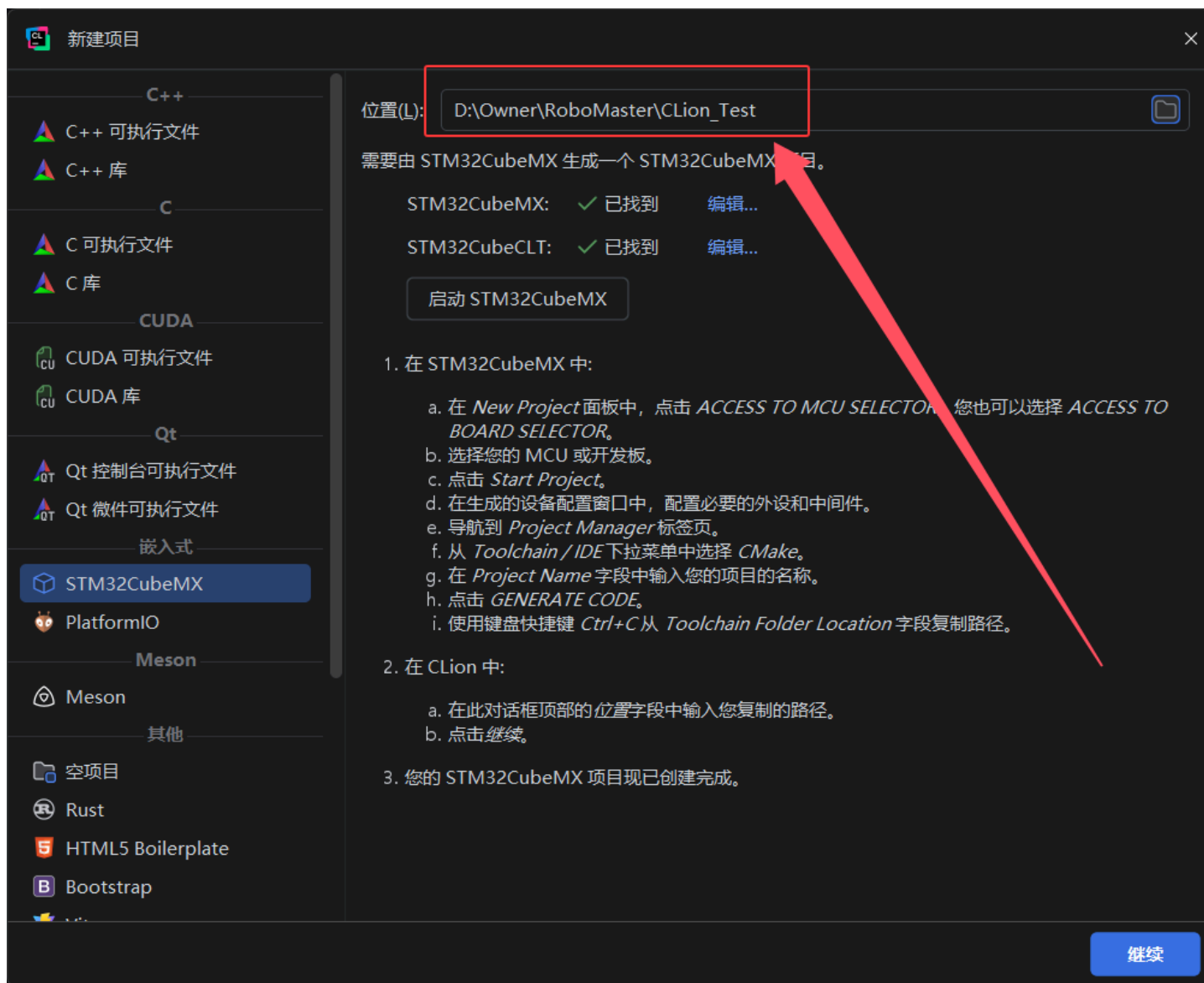


修改Project Name，并且修改好对应的Project Location，并且改动Toolchain/IDE为CMake



到这一步，我们不用去管任何额外的STM32CubeMX设置，直接点击右上角的GENERATE CODE即可。现在，我们要做的只是验证CLion+STM32CubeMX这条工具链的可行性。

跳出生成完毕提示框后，不必Open Folder，直接取消，然后回到CLion，填入我们刚才填写的路径。



只要条件完全满足，右下角的继续就会亮起，然后就进入到我们的CLion配置阶段：

☐ 在编辑 CMakeLists.txt 或其他 CMake 配置文件时重新加载 CMake 项目
外部更改(例如 VCS 更新)会始终触发项目重新加载

配置文件

配置文件是一组命名的构建选项。例如，为调试和发布构建创建单独的配置文件，并在需要时在它们之间切换。

+

—

📁

⬆

⬇

▲ Debug

▲ **Debug - Debug 预设**

▲ RelWithDebInfo 预设

▲ Release 预设

▲ MinSizeRel 预设

▲ RelWithDebInfo - RelWithDebInfo 预设

▲ Release - Release 预设

▲ MinSizeRel - MinSizeRel 预设

☒ 启用配置文件

名称:

Debug - Debug

构建类型:

Debug

▼

对应于 CMAKE_BUILD_TYPE

工具链:

使用默认值 MinGW

▼

生成器:

让 CMake 决定 不显式传递任何生成器

▼

CMake 选项:

所有 CMake 选项 ↗

--preset "Debug" + ↗

> 缓存变量

构建目录:

D:\Owner\RoboMaster\CLion_Test\build\Debug

📁

构建选项:

--preset "Debug" + ↗

'--' 后面的实参传递给此构建，其他实参是 CMake 命令行形参。默认选项取决于工具链的环境。

环境:

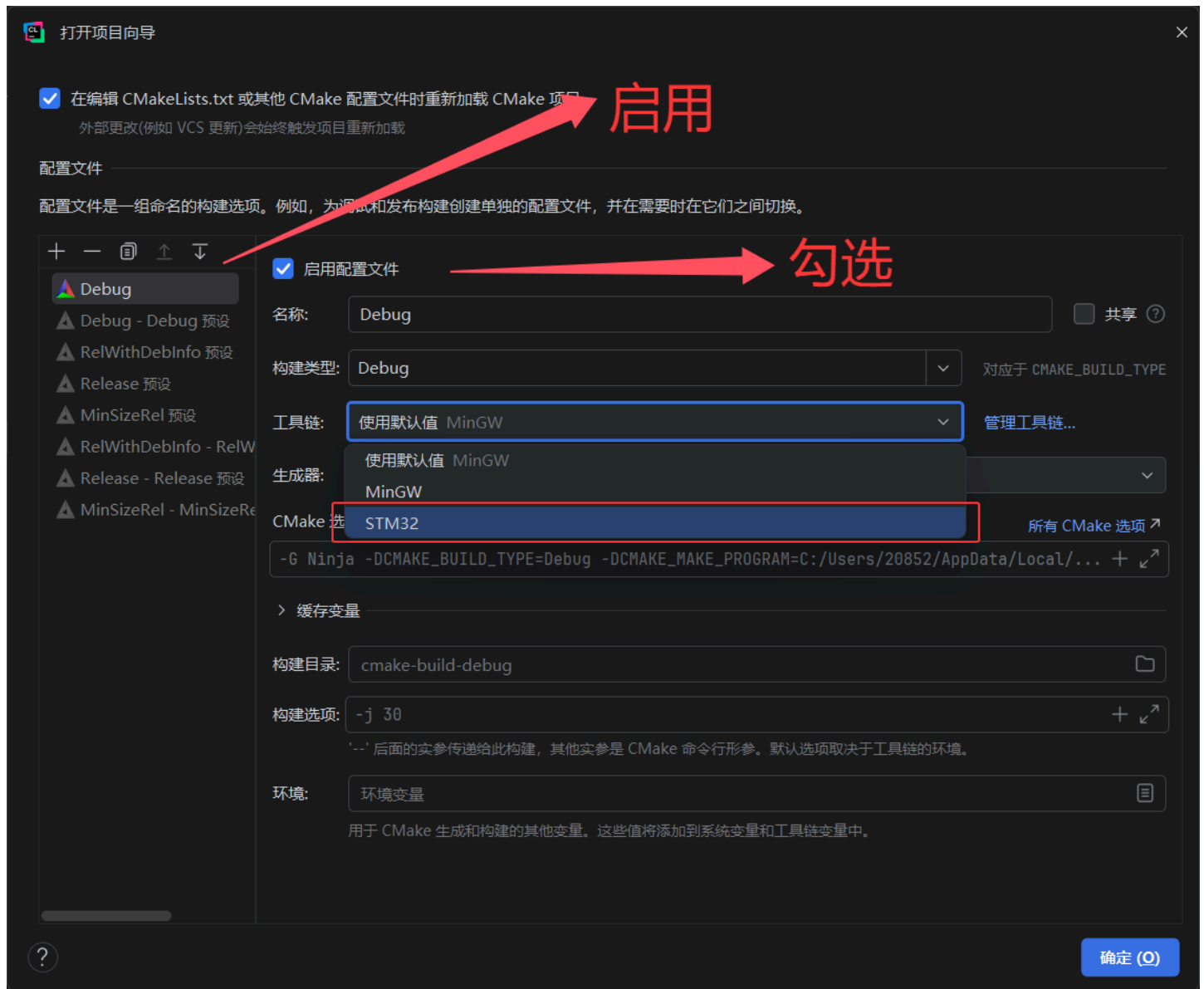
环境变量

📄

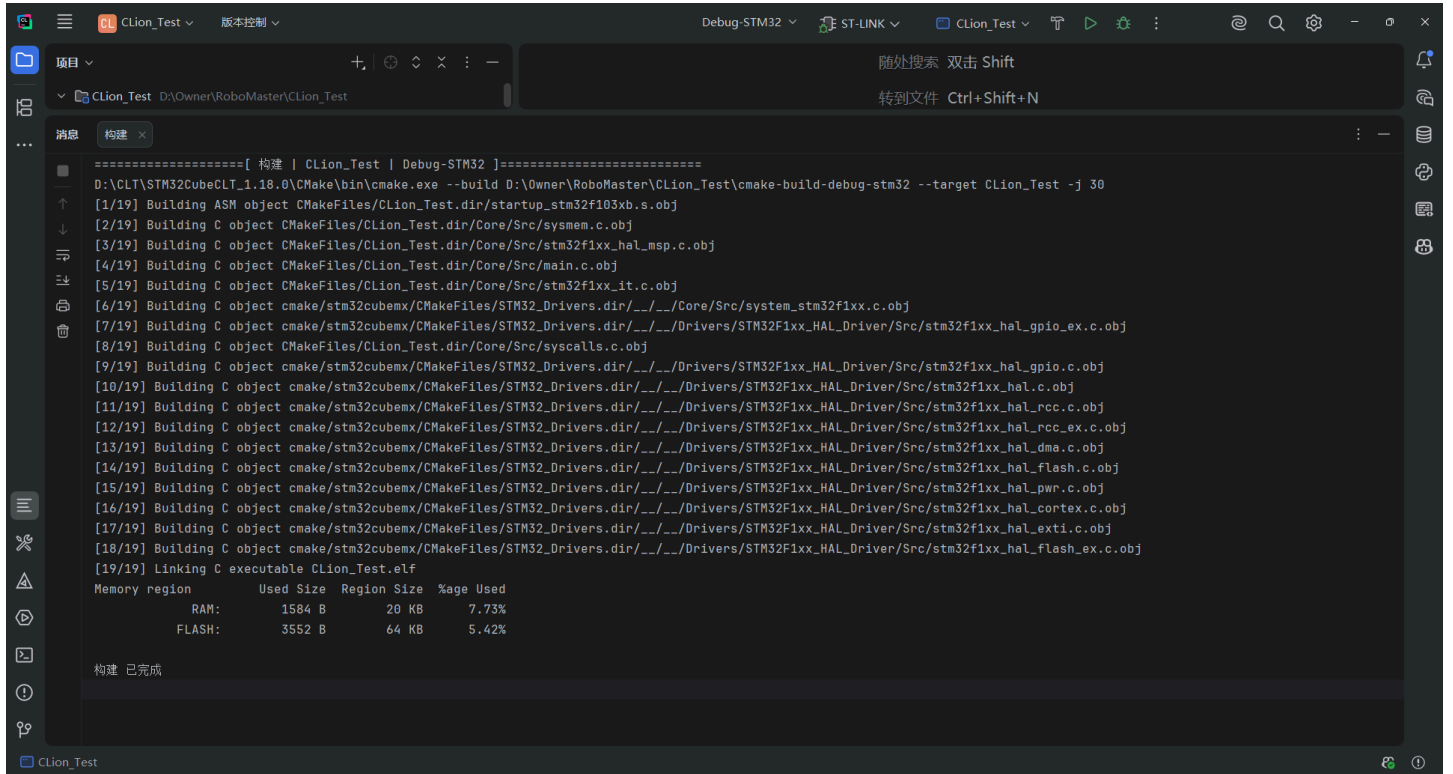
用于 CMake 生成和构建的其他变量。这些值将添加到系统变量和工具链变量中。

取消勾选

不用!



关于图中STM32这条工具链的切换，就需要我们点击右侧的“管理工具链...”，这时就要请出我们的STM32CubeCLT了，左侧加号添加系统工具链，然后按照右侧同样的内容改动即可。（下图版本可忽略）



至此，我们就已经完成了基础的配置。

特殊情况

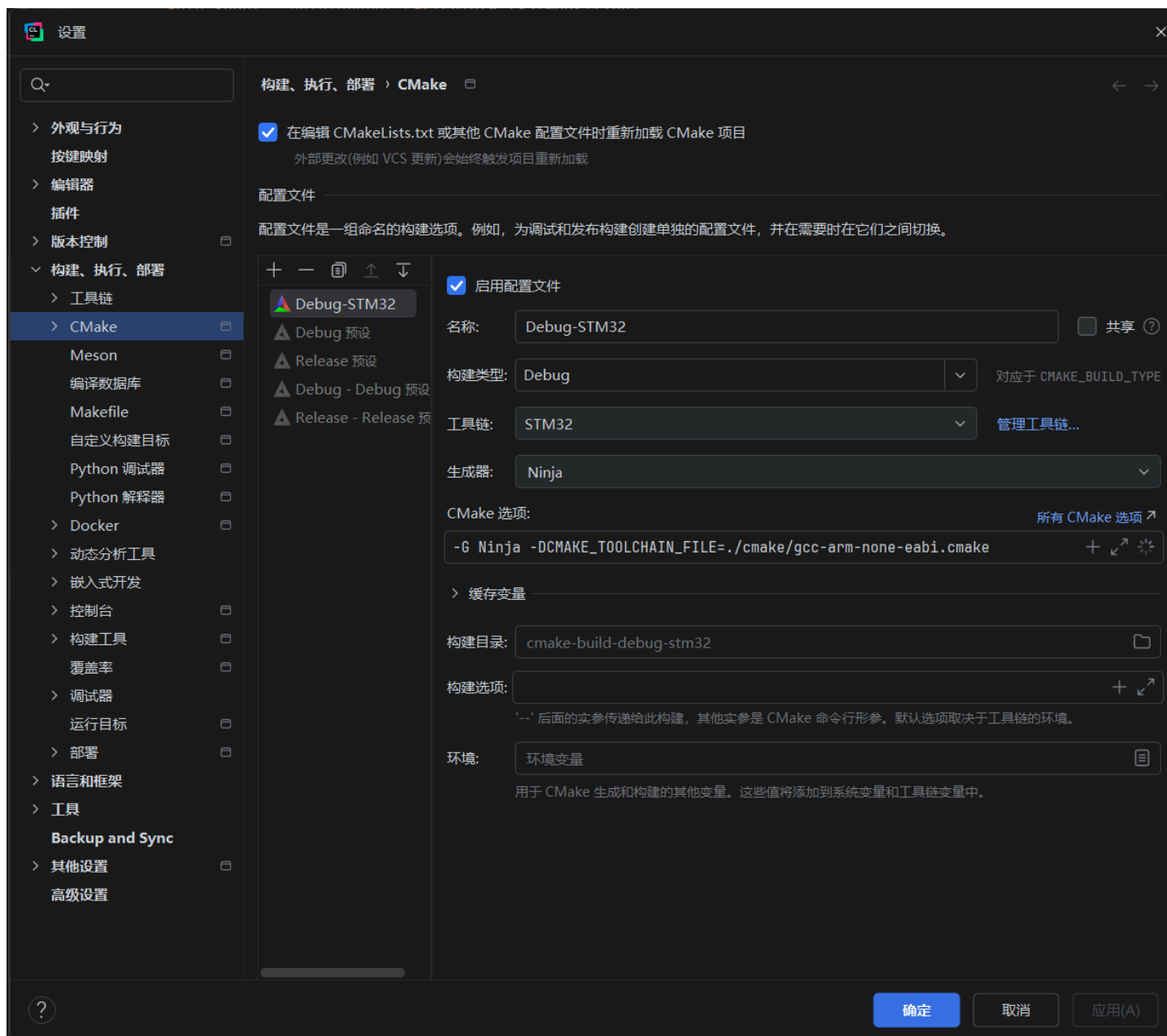
在6.14.1以后的CubeMX版本中，由于新增了对于编译路径的选择，因此在generate code之后有些同学在进行以上操作会后遇到诸如

代码块

```
1 The C compiler
2 "D:/STM32CubeCLT/sTM32CubeCLT_1.22.0/GNU-tools-for-STM32/bin/arm-none-eabi-
  gcc.exe"
3 is not able to compile a simple test program.
```

从英文不难看出，它指的是 C 编译器不能够通过一个简单的编译测试。这指的是我们的 Arm 架构编译 `arm-none-eabi-gcc.exe` 不能够用于编译CLion提供的一个 C 语言编译程序，解决的方式也很简单，只要跳过这一次编译，直接指定好我们下载的编译器即可。

即如下图所示，在CMake选项中添加 `-DCMAKE_TOOLCHAIN_FILE=./cmake/gcc-arm-none-eabi.cmake`即可。



后话：构建完成，然后呢？

至此，我们已经完成了基础工具链的配置。

如果一切顺利，CLion现在已经能够调用STM32CubeMX生成工程，也能够找到STM32CubeCLT提供的编译工具，并最终在下方告诉我们：

构建 已完成。

看到这五个字的时候，你可能会觉得，我们已经成功完成了第一个STM32工程。

但很遗憾，目前为止，我们甚至还没有真正接触到一块STM32开发板。

倘若你手边有开发板，也请千万不要烧录，由于烧录相关内容目前还没有配置，盲目的烧录会导致单片机“成砖”，也就是所谓的烧录锁死，相关内容我们会在后面进行补充讲解。

在前面的配置过程中，我们只是以STM32F103C8T6为例生成了一个最简单的测试工程，并借此确认了下面这条链路能够正常工作：

```
代码块
1 STM32CubeMX生成工程
2     ↓
3   CLion读取工程
4     ↓
5   CMake组织构建过程
6     ↓
7   STM32CubeCLT完成编译
```

换句话说，我们目前只证明了一件事：

电脑已经具备了将C/C++代码转换为STM32程序的能力。

但是，编译成功并不意味着程序已经进入芯片；烧录成功也不意味着程序一定按照我们的想法运行。想要让一段代码真正影响现实中的硬件，我们还缺少后半段流程：

```
代码块
1   完成代码编译
2     ↓
3  通过ST-Link连接开发板
4     ↓
5  将程序烧录到STM32F407
6     ↓
7  芯片从复位位置开始执行
8     ↓
9  GPIO输出发生变化
10    ↓
11  开发板上的LED被点亮
```

在这个过程中，我们还会第一次遇到一些以后经常出现的概念：

- STM32F407到底是一块开发板，还是一颗芯片？
- STM32CubeMX中的时钟树有什么作用？
- 为什么需要启用Serial Wire？
- ST-Link是烧录器还是调试器？
- 编译、烧录和运行分别发生在哪里？
- CubeMX生成的文件分别负责什么？
- HAL库为什么能控制一个真实的引脚？
- 如果LED没有亮，我们应该从哪里开始检查？

这些问题不需要在现在立刻得到完整答案，但我们会从下一篇教程开始，通过一次真正的工程实践逐步认识它们。

下一篇，我们将正式使用本教程的主要入门平台STM32F103C8T6，创建第一个可以编译、烧录、运行并调试的工程。

我们的目标也非常简单：

先不控制电机，也不接收遥控器，让开发板上的一盏LED按照我们的命令闪烁起来。

点灯虽然被大家调侃为嵌入式领域的“Hello World”，但是如果连灯都点不亮，后面的电机大概也不会愿意听我们的话。

当这盏灯真正亮起来时，我们才算完成了从代码到硬件的第一次完整闭环。